INSTITUTO FEDERAL
Goiás
Câmpus Goiânia

Especialização em Inteligência Artificial Aplicada
Disciplina: Deep Learning

Trabalho prático · Frente 3.1 Multilayer Perceptrons

Triagem da gravidade de acidentes em rodovias federais com Multilayer Perceptrons

Classificação multiclasse sobre os dados abertos da PRF (2007 a 2026): modelagem, hiperparametrização com Optuna e avaliação experimental.

Igor Coutinho Ferreira Nobre
Professor: Prof. Dr. Sérgio Canuto

Goiânia, outubro de 2026

Estrutura da apresentação

A apresentação segue a estrutura sugerida no enunciado, nas mesmas seis partes do relatório.

Parte	Conteúdo	Slides
1	Introdução e objetivo do problema	3
2	Descrição da fonte e caracterização do conjunto de dados	4 a 6
3	Arquiteturas, espaço de busca e protocolo de hiperparametrização	7 a 11
4	Resultados de validação e seleção dos modelos promissores	12 a 14
5	Avaliação final no conjunto de teste e análise de custo vs efetividade	15 a 18
6	Limitações e conclusões	19

O slide 20 indica onde cada critério de avaliação é atendido no relatório e na apresentação.

Contexto, objetivo e definição do problema

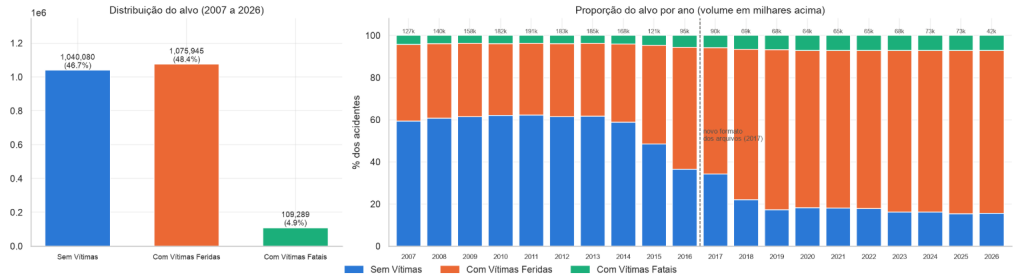
- **Contexto:** quando alguém liga para a PRF relatando um acidente, a central precisa decidir o que enviar com pouca informação: só viatura e guincho, ou também ambulância e resgate.
- **Objetivo:** estimar a gravidade do acidente a partir do que se sabe da ocorrência antes da contagem de vítimas, para ordenar o despacho por risco. O modelo apoia a decisão, não a substitui.
- **Unidade de observação:** um acidente, uma linha dos arquivos anuais da PRF.
- **Variável alvo:** classificação do acidente, definida pela PRF pela consequência mais grave registrada.
- **Tipo de problema:** classificação multiclasse. Frente escolhida: 3.1, Multilayer Perceptrons.

Classe	Significado
Sem Vítimas	ninguém ferido
Com Vítimas Feridas	ao menos um ferido, nenhum morto
Com Vítimas Fatais	ao menos um morto

As três classes são mutuamente exclusivas.

Fonte, tamanho e distribuição do alvo

- **Fonte:** "Acidentes nas Rodovias Federais (agrupados por ocorrência)", PRF, Portal Brasileiro de Dados Abertos. Um CSV por ano, 2007 a 2026.
- **Tamanho:** 2.237.189 ocorrências. Removidos 0,53% (alvo ausente ou ignorado e 6 duplicatas): restam 2.225.314.
- **Entradas:** 12 variáveis categóricas em três grupos: contexto da via, quando e onde, relato do solicitante. No máximo 70 ausentes por coluna.
- **Alvo desbalanceado:** 48,4% com feridos, 46,7% sem vítimas e 4,9% com fatais. A classe mais rara é a que mais importa para o despacho.
- **Hora e dia importam:** a taxa de fatais chega a 9% na madrugada contra 3% entre 7 h e 17 h; sábado 5,8% e domingo 6,7% contra 4,3% nos demais dias. Daí as entradas derivadas período do dia e fim de semana.



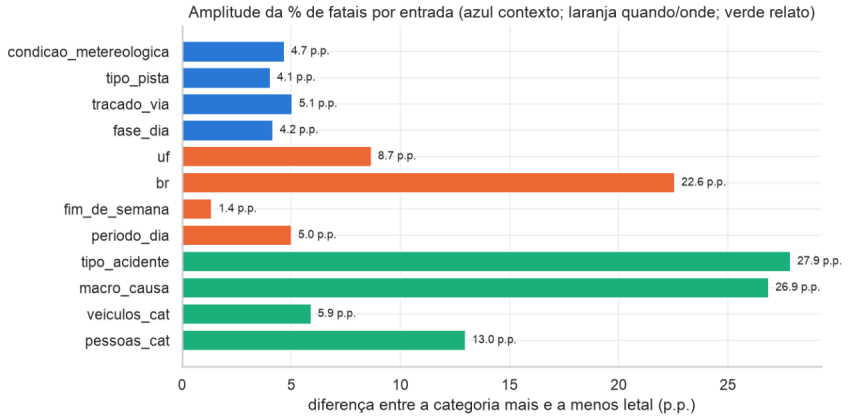
Distribuição do alvo e proporção por ano. Fonte: elaborado pelo autor.



Volume e taxa de fatais por hora e dia da semana.

Seleção de atributos e risco de vazamento

- **Excluídas por vazamento direto:** as contagens de mortos, feridos leves e graves e ilesos definem o rótulo. Todo acidente fatal tem mortos maior que zero.
- **Excluídos por viés de registro:** o ano e os campos que só existem a partir de 2017. A fração de Sem Vítimas cai de 60% até 2014 para 16% a partir de 2019 por mudança no procedimento da PRF, não no perfil dos acidentes. Com o ano, a rede aprenderia a época, não a gravidade.
- **Agrupada:** a causa detalhada, cujo vocabulário cresce de 11 valores em 2016 para 75 em 2023, virou 7 macrocausas. Ressalva: a causa é avaliação da equipe, não do solicitante.
- **Excluídas por cardinalidade ou pouca informação:** km, município e sentido da via, depois de UF e BR.
- **O que mais separa as classes é o relato:** a taxa de fatais varia 27,9 pontos percentuais entre os tipos de acidente, contra 4 a 5 pontos nas variáveis de contexto da via.



Amplitude da taxa de fatais por entrada: categoria mais letal contra a menos letal. Fonte: elaborado pelo autor.

Divisão dos dados e pré-processamento

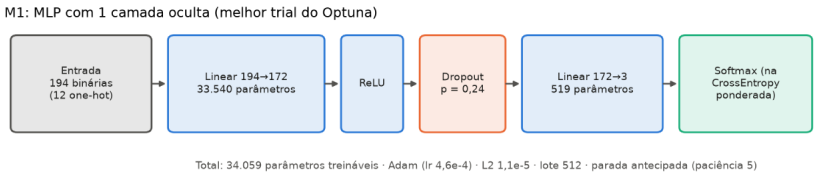
- **Divisão 70 / 15 / 15:** 1,56 milhão de treino, 334 mil de validação e 334 mil de teste, aleatória e estratificada pelo alvo. Cada linha é um acidente distinto, sem unidades repetidas.
- **Codificação one-hot:** cada categoria vira uma coluna 0 ou 1, porque a rede só recebe números. São 194 colunas. Categorias com menos de 200 ocorrências ou nunca vistas caem numa coluna única.
- **Sem vazamento entre partições:** o encoder é ajustado só no treino e depois aplicado à validação e ao teste. Entradas binárias dispensam normalização.
- **Desbalanceamento:** pesos de classe na função de perda, calculados no treino e amortecidos pela raiz quadrada (0,845 / 0,830 / 2,605). O peso integral, cerca de 6,8 para fatais, faria o modelo prever fatal em excesso.
- **Validação cruzada** não foi usada pelo custo com 2,2 milhões de linhas.

Partição	Finalidade neste trabalho
Treino	ajusta os pesos da rede e o encoder
Validação	escolhe hiperparâmetros, parada antecipada e modelo final
Teste	estima o desempenho final; não participa de nenhuma seleção

Mesmo papel das partições pedido no enunciado (Seção 2.3).

Arquitetura da MLP e treinamento

- **Implementação própria em PyTorch:** sequência de blocos Linear, ReLU e Dropout. Linear combina as entradas, ReLU corta os valores negativos, Dropout desliga neurônios ao acaso durante o treino.
- **Entrada:** 194 neurônios, as colunas one-hot. **Ocultas:** 1 a 3 camadas, com largura definida pela busca.
- **Saída:** 3 neurônios, um por classe. A softmax converte as saídas em probabilidades que somam 1; a entropia cruzada ponderada dá mais peso à classe rara.
- **Treinamento:** retropropagação em mini-batches, otimizador Adam, inicialização padrão do PyTorch. Uma única função de treino é usada na busca e nos modelos finais.
- **Regularizações disponíveis:** dropout, L2 (weight decay), L1 somado à perda e parada antecipada pelo Macro F1 de validação, que restaura a melhor época. BatchNorm avaliado à parte.



Arquitetura do modelo final (M1): 194 entradas, uma camada de 172 neurônios, 3 saídas. Fonte: elaborado pelo autor.

Termos usados nos resultados

- **Época e lote:** época é uma passagem por todo o treino; lote (mini-batch) é a fatia usada em cada passo do gradiente. Lotes menores dão passos ruidosos, que regularizam; maiores aceleram.
- **Taxa de aprendizado:** tamanho do passo do otimizador. Baixa demais, a rede não converge no orçamento de épocas; alta demais, oscila.
- **Sobreajuste, subajuste e gap:** sobreajuste é decorar o treino e piorar na validação; subajuste é não aprender nem o treino. O gap, treino menos validação, é o termômetro.
- **Dropout:** desliga ao acaso uma fração dos neurônios a cada passo. A rede deixa de depender de poucos neurônios e sobreajusta menos.
- **L1 e L2:** penalidades somadas à perda pelo tamanho dos pesos. L2 encolhe todos; L1 zera alguns. Reduzem o gap; em excesso, subajustam.
- **BatchNorm:** normaliza a saída de cada camada dentro do lote. Acelera a otimização; não é regularização por si.
- **Parada antecipada:** acompanha o Macro F1 de validação e para quando ele não melhora por N épocas (a paciência), restaurando a melhor época.
- **Macro F1:** média do F1 das três classes, com o mesmo peso. Faz a classe fatal, 5% dos dados, contar tanto quanto as outras. É a métrica principal.
- **Pesos de classe:** multiplicam a perda dos exemplos da classe rara. Deslocam o ponto de operação: mais recall de fatais, menos precisão.
- **Busca bayesiana (TPE):** escolhe a próxima combinação de hiperparâmetros usando os resultados das anteriores. Explicada no próximo slide.

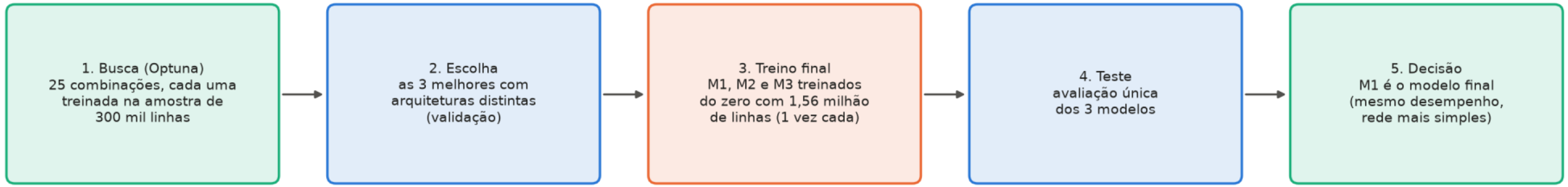
Por que hiperparametrizar e como funciona a busca bayesiana

- **Parâmetros** são os pesos, que a rede aprende. **Hiperparâmetros** (taxa de aprendizado, largura, profundidade, lote, dropout, L1, L2) são fixados antes do treino e decidem desempenho, generalização e custo.
- **Grid search** percorre toda a grade: 7 hiperparâmetros com 5 valores cada dariam 78.125 treinos. Inviável.
- **Random search** sorteia combinações. Cobre mais que a grade com o mesmo orçamento, mas não aprende com o que já testou.
- **Busca bayesiana** usa as tentativas anteriores para escolher a próxima. Foi a estratégia adotada, com o Optuna e o amostrador TPE.

Como o TPE escolhe a próxima tentativa

1. Começa com 5 tentativas aleatórias.
2. Separa as tentativas feitas em boas e ruins pelo Macro F1 de validação.
3. Estima em que região do espaço ficam as boas e em que região ficam as ruins.
4. Testa o ponto onde é mais provável ser bom e menos provável ser ruim.
5. Um pruner interrompe, a partir da 5ª época, as tentativas abaixo da mediana das anteriores, para não gastar tempo com elas.

Protocolo: da busca de hiperparâmetros ao modelo final



Da busca ao modelo final. Fonte: elaborado pelo autor.

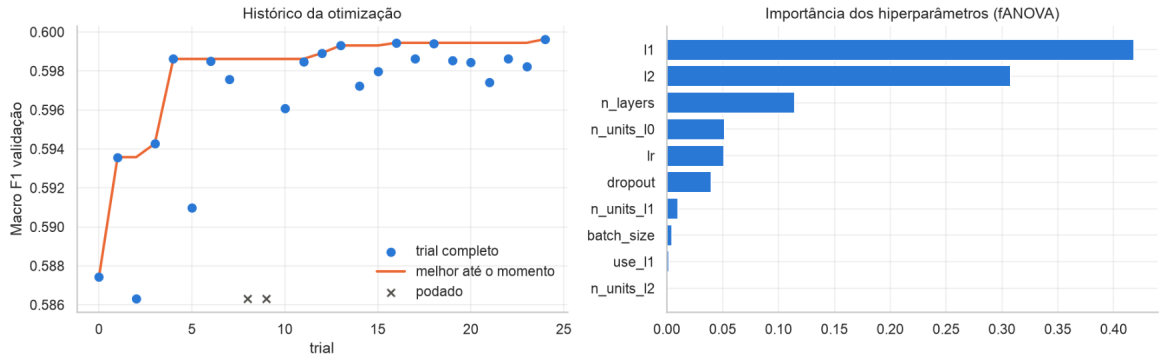
- **Busca:** 25 tentativas numa subamostra estratificada de 300 mil linhas do treino, até 15 épocas, Macro F1 medido na validação completa. A busca só ranqueia as combinações.
- **Experimentos controlados:** largura × profundidade e regularização técnica por técnica, com o resto fixo, para isolar o efeito de cada escolha.
- **Finalistas:** as 3 melhores arquiteturas distintas, treinadas do zero com toda a base de treino, até 30 épocas, parada antecipada com paciência 5.
- **Teste:** uma avaliação por finalista. A seleção é feita na validação; em empate, decide o custo. A subamostra manteve o ranqueamento: 0,600 na busca, 0,605 com a base inteira.

Espaço de busca e configuração do estudo no Optuna

Hiperparâmetro	Intervalo	Justificativa
Taxa de aprendizado	1e-4 a 5e-3, escala log	em torno do padrão do Adam, 1e-3
Camadas ocultas	1 a 3	uma basta em princípio; mais camadas podem usar menos neurônios
Neurônios por camada	16 a 256, escala log	de compressão forte a 1,3 vez as 194 entradas
Tamanho do lote	256, 512, 1024, 2048	lotes menores regularizam, maiores aceleram
Dropout	0 a 0,5	faixa usual, com a opção de desligar
L2	1e-6 a 1e-2, log	acima disso a penalidade supera a entropia cruzada
L1	desligado ou 1e-7 a 1e-4, log	mesmo critério do L2

- **Tentativas:** 25, orçamento definido pelo tempo em CPU; as melhores convergem para a mesma região.
- **Amostrador:** TPE com semente fixa, para reprodutibilidade.
- **Poda:** a partir da 5ª época, tentativas abaixo da mediana são interrompidas.
- **Dados:** subamostra de 300 mil linhas do treino, até 15 épocas; avaliação na validação completa.
- **Métrica otimizada:** Macro F1 de validação, que dá o mesmo peso às três classes.
- **Fora da busca:** otimizador, ativação e inicialização, por baixa influência esperada numa MLP rasa com entradas binárias.

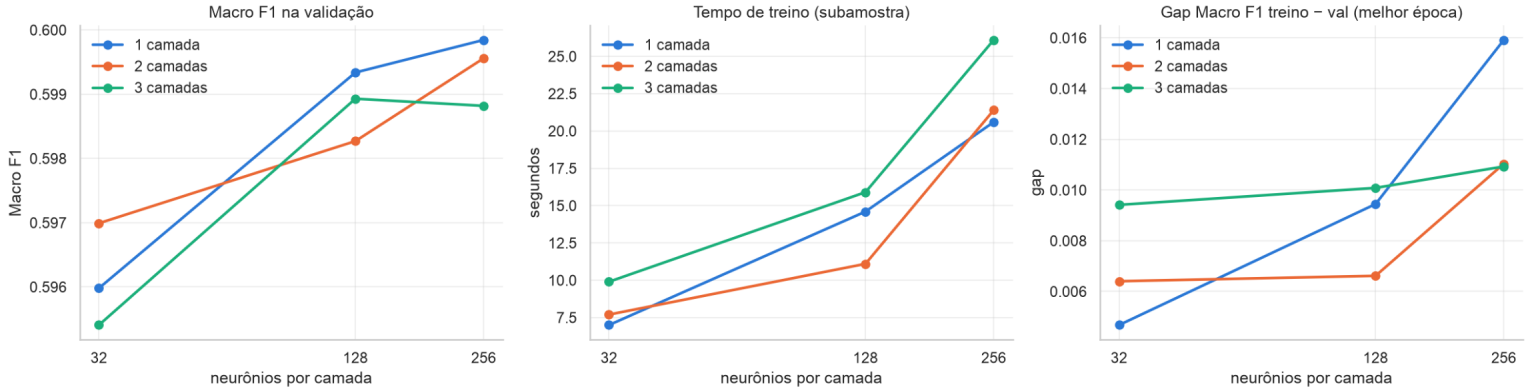
Resultados da busca de hiperparâmetros



Histórico da busca e importância dos hiperparâmetros (fANOVA). Fonte: elaborado pelo autor.

- **Tempo:** 6,2 minutos em CPU para as 25 tentativas. Em 2 delas o pruner interrompeu o treino na 5ª época por estarem abaixo da mediana das demais.
- **Melhor tentativa:** Macro F1 de 0,5996 na validação, com uma camada de 172 neurônios, dropout 0,24 e lote 512. A busca convergiu para redes rasas e largas.
- **Empate no topo:** as 17 melhores ficam entre 0,597 e 0,600, diferença menor que o ruído de treino. Não dá para dizer que uma é melhor que a outra.
- **O que mais prejudica:** as piores tentativas têm L2 acima de 0,001, cem vezes o dos finalistas, com taxa de aprendizado baixa. A rede encolhe os pesos demais e subajusta.
- **Como ler o gráfico de importância:** mede quanto da variação do Macro F1 entre tentativas cada hiperparâmetro explica. L1 e L2 lideram porque estragaram as piores tentativas, não porque mudem o resultado perto do ótimo.

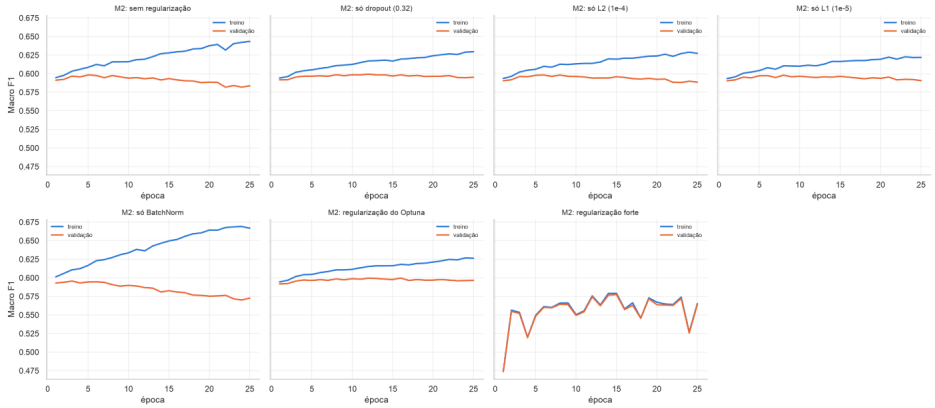
Efeito da largura e da profundidade



Experimento controlado: 3 profundidades × 3 larguras, resto fixo, subamostra de 300 mil linhas. Fonte: elaborado pelo autor.

- **Largura:** com 32 neurônios a rede é pequena para o problema, subajuste leve (0,595 a 0,597). Subir para 128 resolve; de 128 para 256 quase nada muda.
- **Profundidade:** empilhar 2 ou 3 camadas não melhora a validação. Redes mais fundas atingem o melhor ponto na época 4 ou 5 e depois só aumentam o gap, ou seja, decoram o treino.
- **Custo:** a rede de 3 camadas com 256 neurônios tem 182 mil parâmetros, 3,6 vezes a de 1 camada com 256 (51 mil), e não é melhor.
- **Interpretação:** as entradas são 194 colunas binárias e a relação com a gravidade é quase aditiva; uma camada já captura as interações que importam. A diferença entre a melhor e a pior das 9 redes é de 0,0044 de Macro F1: a arquitetura não é o gargalo deste problema.

Comparação das estratégias de regularização



Macro F1 de treino e validação por época. Arquitetura de M2, subamostra de 300 mil, 25 épocas sem parada antecipada.

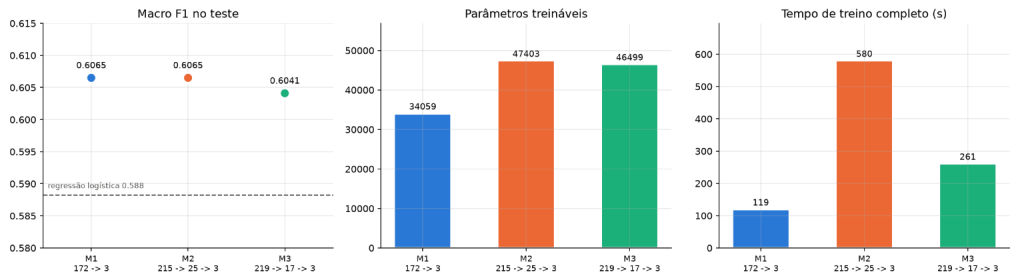
Configuração	Melhor val.	Gap	Diagnóstico
Sem regularização	0,598	0,060	sobreajuste
Só dropout 0,32	0,599	0,034	isolada mais eficaz
Só L2 1e-4	0,598	0,039	reduz o gap, val. cai
Só L1 1e-5	0,598	0,031	parecido com L2
Só BatchNorm	0,596	0,094	antecipa o sobreajuste
Combinação do Optuna	0,600	0,030	melhor val., gap pequeno
Forte: dropout 0,5, L2 1e-2	0,578	0,001	subajuste

- **Sem regularização** a rede memoriza: o treino sobe sem parar e a validação cai depois da 5ª época. **Dropout** sozinho é a técnica mais eficaz; a combinação do Optuna é essencialmente esse dropout.
- **L1 e L2** reduzem o gap sem melhorar a validação. **BatchNorm** sozinho acelera a otimização e antecipa o sobreajuste; não é regularização por si. **Regularização forte** zera o gap, mas subajusta.
- **Parada antecipada** simulada sobre as curvas: paciência 1 para cedo e perde até 0,008; paciência 4 ou 5 recupera o melhor valor em todas as configurações. Por isso 4 na busca e 5 no treino final.

Desempenho e custo dos três finalistas no teste

Modelo	Arquitetura	Parâmetros	Treino (s)	Inferência (s)	Acurácia	Macro F1	F1 val.
M1	194 → 172 → 3	34.059	119	0,13	0,7029	0,6065	0,6046
M2	194 → 215 → 25 → 3	47.403	580	0,10	0,7029	0,6065	0,6041
M3	194 → 219 → 17 → 3	46.499	261	0,10	0,7023	0,6041	0,6017

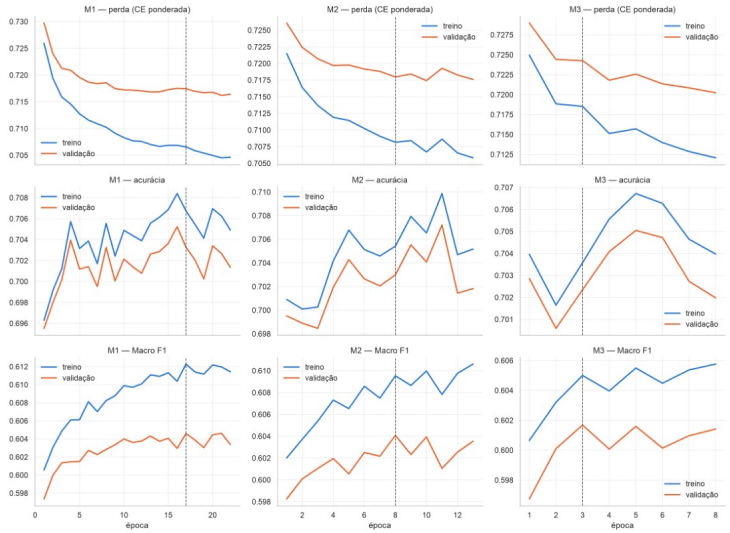
Inferência sobre os 333.798 exemplos do teste, em CPU. Treino com toda a base de treino, até 30 épocas, paciência 5, uma semente por modelo.



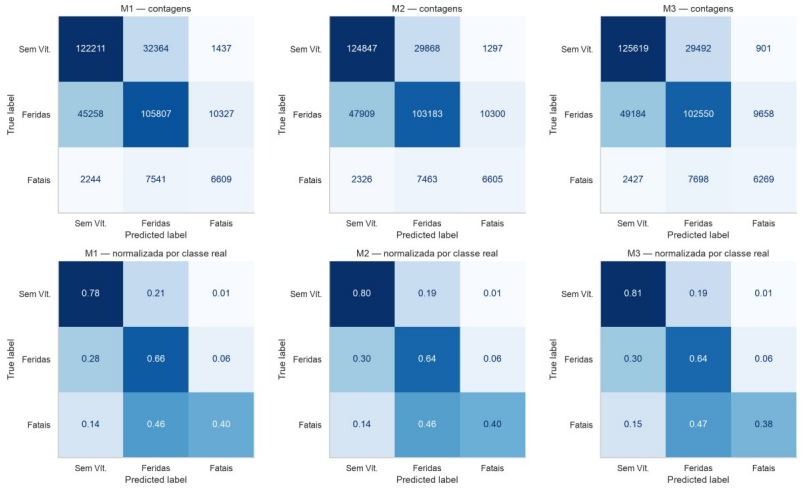
Macro F1 no teste, parâmetros e tempo de treino. Fonte: elaborado pelo autor.

- **Finalistas:** as três melhores tentativas da busca com arquiteturas distintas.
- **Resultado:** M1 e M2 empatam no teste; M3 fica 0,002 abaixo. Com uma semente por modelo, diferenças dessa ordem não são significativas.
- **Escolha:** M1, que já tinha o melhor Macro F1 de validação, com 28% menos parâmetros que M2 e um quinto do tempo de treino. A camada extra de M2 e M3 não trouxe ganho.
- **Custo de uso:** 0,1 s para classificar 334 mil acidentes em CPU, irrelevante para a central.

Curvas de treino e validação e matriz de confusão do M1



Perda, acurácia e Macro F1 por época; a linha tracejada marca a melhor época.



Matrizes de confusão do M1 no teste, em contagens e normalizadas pela classe real.

- **Convergência:** as perdas de treino e validação caem juntas. Quando o Macro F1 de validação para de subir, a parada antecipada guarda a melhor época (17 no M1, 8 no M2, 3 no M3).
- **Sobreajuste leve:** treino e validação ficam a menos de 0,01 um do outro na melhor época. **Sem subajuste:** aumentar a capacidade não melhora a validação.
- **Matriz:** cada linha é a classe real, cada coluna a prevista. Sem Vítimas é a mais fácil, 78% acertados. O erro mais comum é trocar Sem Vítimas por Com Feridos, que têm relatos parecidos.
- **Classe fatal:** 40% reconhecidos, 46% vão

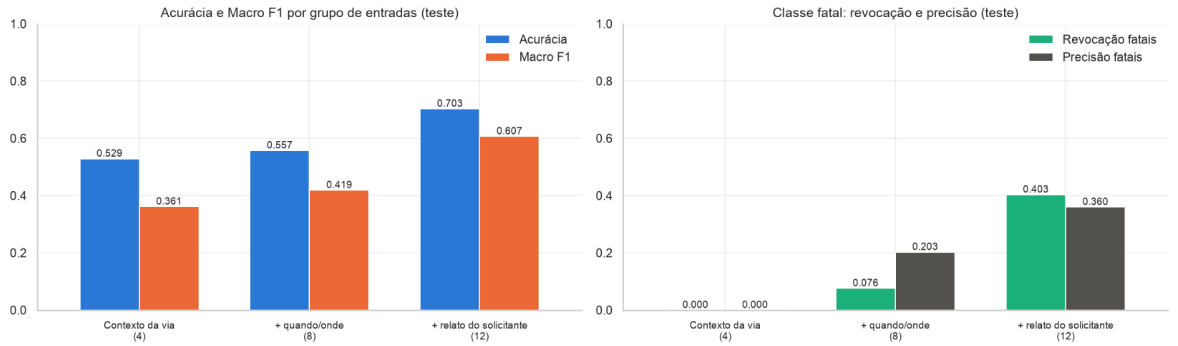
Exemplo: como ler as métricas da classe fatal

Métrica	Valor	Leitura
Recall	0,40	de 100 acidentes fatais reais, 40 recebem o rótulo fatal
Precisão	0,36	de 100 rotulados como fatais, 36 são fatais de verdade
F1	0,38	média harmônica: 2 × 0,40 × 0,36 ÷ (0,40 + 0,36)
Macro F1	0,607	média do F1 das três classes, com o mesmo peso

- **Os 60 que passam:** 46 vão para Com Feridos e 14 para Sem Vítimas. Como 86 em 100 fatais são sinalizados como "com vítima", o socorro é acionado na maioria dos casos.
- **O erro grave são os 14** rotulados como Sem Vítimas, que não acionam socorro.
- **Por que a média harmônica:** o F1 cai quando um dos dois, recall ou precisão, é baixo. Não dá para inflar um sacrificando o outro.
- **Por que Macro F1 e não acurácia:** com 5% de fatais, um modelo que nunca prevê fatal tem 48% de acurácia e parece razoável. O Macro F1 dá à classe fatal o mesmo peso das outras.
- **Ponto de operação:** esse recall vem dos pesos de classe. Um peso maior para fatais elevaria o recall à custa da precisão. O modelo prioriza; não decide sozinho.

Comparação com baselines e ablação por grupo de entradas

Modelo no teste	Acurácia	Macro F1
Classe majoritária	0,484	0,217
Sorteio proporcional às classes	0,455	0,334
Tabela de frequências, só contexto da via	0,529	0,361
Regressão logística, 12 entradas	0,685	0,588
MLP M1	0,703	0,607



Arquitetura de M1 treinada com grupos crescentes de entradas, no teste. Fonte: elaborado pelo autor.

- **Baselines:** a tabela de frequências prevê, para cada combinação das 4 entradas de contexto, a classe mais comum no treino; é o teto do que essas entradas permitem. A regressão logística é linear e usa as mesmas 12 entradas da MLP.
- **Ganho da MLP:** 1,8 p.p. de acurácia e de Macro F1 sobre a regressão logística. Vem das interações entre entradas que só a camada oculta captura. Real, mas pequeno.
- **Ablação:** só com o contexto da via, 52,9% e nenhum fatal previsto; local e horário somam 2,9 p.p.; o relato do solicitante soma 14,6 p.p. e leva o recall dos fatais de 0 para 0,40. As entradas pesam muito mais que a arquitetura.

Limitações e conclusões

Conclusões

- Uma MLP com uma camada de 172 neurônios (34 mil parâmetros) classifica a gravidade com 70,3% de acurácia e Macro F1 de 0,607 no teste, contra 48% da classe majoritária e 68,5% da regressão logística.
- Perto do ótimo, o desempenho é pouco sensível à arquitetura; regularização forte é o que mais prejudica. Largura pesa até cerca de 128 neurônios; mais camadas não ajudam neste problema tabular.
- Dropout foi a regularização mais eficaz; BatchNorm sozinho só antecipou o sobreajuste; parada antecipada com paciência 4 ou 5 protegeu o resultado.
- O limite está nas entradas, não na rede: o relato responde por cerca de 15 pontos de acurácia.

Limitações e trabalhos futuros

- **Classe fatal:** 40% de recall é o ponto de operação dos pesos de classe; o compromisso com a precisão não foi explorado. Serve para priorizar, nunca para negar socorro.
- **Dois regimes de registro** misturados no teste aleatório, e a causa provável vem da equipe, não do solicitante. Um teste cronológico e a ablação por variável mediriam melhor o uso real.
- **Uma semente e uma divisão:** sem intervalo de confiança; diferenças de 0,002 não são significativas.
- **Próximos passos:** várias sementes, classes ordinais, limiar da classe fatal pelo custo de cada erro.

Onde cada critério é atendido

Critério	O que o trabalho faz	Onde
I · Justificativas das escolhas (3 pts)	Exclusão de vítimas e ano por vazamento e viés; macrocausas; one-hot ajustado no treino; divisão estratificada; pesos de classe com raiz; cada intervalo do espaço de busca; Macro F1 como métrica; paciência da parada verificada	Relatório, caps. 2 e 3 · slides 4 a 11
II · Corretude da modelagem (2 pts)	MLP construída do zero em PyTorch: 194 entradas, blocos Linear, ReLU, Dropout, 3 saídas com softmax na perda ponderada; Adam; L1 só nos pesos; parada antecipada restaurando a melhor época; sem vazamento entre partições	Relatório, cap. 3 e Código 1 · slide 7
III · Hiperparametrização (2 pts)	Optuna com TPE e poda, 25 tentativas, espaço de 7 hiperparâmetros justificado, métrica só na validação; registro completo em CSV; grid controlado de largura e profundidade; regularização técnica por técnica; 3 finalistas	Relatório, caps. 3 e 4 · slides 9 a 14
IV · Interpretação dos resultados (2 pts)	Curvas de treino e validação, gap e diagnóstico de sub e sobreajuste; matriz de confusão e leitura da classe fatal; custo versus efetividade; baselines e ablação por grupo de entradas; limitações declaradas	Relatório, caps. 4, 5 e 6 · slides 14 a 19

Entregáveis: relatório técnico em PDF; repositório no GitHub com o notebook; registro da busca em optuna_trials.csv e config_final.json; tabelas e figuras comparativas das arquiteturas.

Obrigado.

Notebook, registro da busca de hiperparâmetros, artefatos e relatório estão no repositório:

github.com/coutinhonobre/Trabalho-Final---Pos-Deep-Learning

Igor Coutinho Ferreira Nobre

Especialização em Inteligência Artificial Aplicada · IFG Goiânia